

## **Program 1: WAP to demonstrate Diffie Hellman cryptosystem.**

**Code:**

```
# PROGRAM 2: Diffie-Hellman Key Exchange Demonstration
```

```
def power_mod(base, exp, mod):
```

```
    return pow(base, exp, mod)
```

```
# Public values
```

```
p = int(input("Enter a prime number p: "))
```

```
g = int(input("Enter a primitive root g: "))
```

```
# Private keys
```

```
a = int(input("Enter private key for User A: "))
```

```
b = int(input("Enter private key for User B: "))
```

```
# Public keys
```

```
A = power_mod(g, a, p)
```

```
B = power_mod(g, b, p)
```

```
print("\nPublic key of A:", A)
```

```
print("Public key of B:", B)
```

```
# Shared secret
```

```
shared_A = power_mod(B, a, p)
```

```
shared_B = power_mod(A, b, p)
```

```
print("\nShared secret computed by A:", shared_A)
print("Shared secret computed by B:", shared_B)

if shared_A == shared_B:
    print("\nKey Exchange Successful! Shared Secret =", shared_A)
else:
    print("\nError: Shared secrets do not match!")
```

### **Output:**

**Enter a prime number p: 7**

**Enter a primitive root g: 11**

**Enter private key for User A: 77**

**Enter private key for User B: 154**

**Public key of A: 2**

**Public key of B: 4**

**Shared secret computed by A: 2**

**Shared secret computed by B: 2**

**Key Exchange Successful! Shared Secret = 2**

## Program 2: WAP to demonstrate ECC cryptosystem.

### Code:

```
# PROGRAM 3: ECC Cryptosystem Demonstration

# Uses Elliptic Curve:  $y^2 = x^3 + ax + b \pmod{p}$ 

def inv_mod(k, p):
    return pow(k, p-2, p)

def point_add(P, Q, a, p):
    if P == (0, 0):
        return Q
    if Q == (0, 0):
        return P
    if P == Q:
        s = ((3 * P[0] * P[0] + a) * inv_mod(2 * P[1], p)) % p
    else:
        s = ((Q[1] - P[1]) * inv_mod(Q[0] - P[0], p)) % p
    xr = (s * s - P[0] - Q[0]) % p
    yr = (s * (P[0] - xr) - P[1]) % p
    return (xr, yr)

def scalar_mult(k, P, a, p):
    R = (0, 0) # Identity
    while k > 0:
        if k & 1:
            R = point_add(R, P, a, p)
        P = point_add(P, P, a, p)
        k >>= 1
```

```

    return R

# -----

# Main Program Inputs
# -----

p = int(input("Enter prime p: "))
a = int(input("Enter curve parameter a: "))
b = int(input("Enter curve parameter b: "))
print("\nEnter base point G (x, y):")
Gx = int(input("Gx: "))
Gy = int(input("Gy: "))
G = (Gx, Gy)
private_key = int(input("Enter private key: "))

# Public key
public_key = scalar_mult(private_key, G, a, p)
print("\nPublic Key:", public_key)

print("\nEnter message point M (x, y):")
Mx = int(input("Mx: "))
My = int(input("My: "))
M = (Mx, My)
k = int(input("Enter random value k: "))

# Encryption
C1 = scalar_mult(k, G, a, p)

```

```
C2 = point_add(M, scalar_mult(k, public_key, a, p), a, p)
print("\nEncrypted Points:")
print("C1:", C1)
print("C2:", C2)
```

### **Output:**

**Enter prime p: 11**

**Enter curve parameter a: 15**

**Enter curve parameter b: 30**

**Enter base point G (x, y):**

**Gx: 44**

**Gy: 88**

**Enter private key: 55**

**Public Key: (44, 88)**

**Enter message point M (x, y):**

**Mx: 66**

**My: 77**

**Enter random value k: 90**

**Encrypted Points:**

**C1: (0, 0)**

**C2: (66, 77)**